

ui.menu 全面详解

ui.menu 是 NiceGUI 基于 Quasar 的 QMenu 组件封装的上下文菜单 / 下拉菜单元素，支持点击触发、右键触发、嵌套子菜单等核心功能，广泛适用于界面操作入口、功能导航、上下文快捷操作等场景。其核心特性包括灵活的触发方式、多级菜单嵌套、样式自定义、事件联动等，可快速实现交互友好的菜单导航体验。

一、核心概念与基础特性

1. 本质与用途

- 本质：封装 Quasar 的 QMenu 组件，通过嵌套 `ui.menu_item`（菜单项）、`ui.menu_divider`（菜单分隔线）、`ui.submenu`（子菜单）定义菜单结构，需绑定到触发元素（如按钮、图标）或通过右键触发。
- 核心用途：提供隐藏式操作入口，节省界面空间，常见于顶部导航下拉菜单、表格行右键菜单、功能按钮关联菜单等场景。
- 关键机制：默认通过点击触发元素显示菜单，支持配置为右键触发；菜单显示位置自动贴合触发元素，支持手动调整偏移；支持多级子菜单嵌套，点击子菜单触发元素自动展开下级菜单。

2. 基础结构

一个完整的 ui.menu 由触发元素、菜单容器、菜单项 / 子菜单 / 分隔线组成，示例结构如下：

```
from nicegui import ui

# 基础点击触发菜单
with ui.button('操作菜单') as trigger: # 菜单触发元素
    with ui.menu(trigger=trigger): # 绑定触发元素
        ui.menu_item('新建', on_click=lambda: ui.notify('新建文件')) # 普通菜单项
        ui.menu_item('打开', on_click=lambda: ui.notify('打开文件'))
        ui.menu_divider() # 菜单分隔线
        with ui.submenu('导出'): # 子菜单
            ui.menu_item('导出为 PDF', on_click=lambda: ui.notify('导出 PDF'))
            ui.menu_item('导出为 Excel', on_click=lambda: ui.notify('导出 Excel'))
        ui.menu_item('退出', on_click=lambda: ui.notify('退出程序')).props('color=red') # 自定义样式

# 右键触发菜单（无显式触发元素，绑定到页面）
with ui.menu(trigger=ui.page, on_context_menu=True):
    ui.menu_item('复制', on_click=lambda: ui.notify('复制内容'))
    ui.menu_item('粘贴', on_click=lambda: ui.notify('粘贴内容'))

ui.run()
```

二、初始化配置项

初始化 `ui.menu()` 时可通过参数配置触发方式、显示位置、行为等核心属性，参数说明如下：

参数名	类型	说明
trigger	Element Page	菜单触发元素（必填），可绑定到任意 NiceGUI 元素（如按钮、图标）或 <code>ui.page</code> （页面）
on_context_menu	bool	是否通过右键点击触发菜单（默认 False，即左键点击触发；设为 True 时右键触发）
value	bool	初始显示状态（默认 False，即隐藏；设为 True 时初始显示菜单）
on_value_change	Callable	菜单显示 / 隐藏状态变化时触发的回调函数（接收 <code>ValueChangeEventArguments</code> 参数， <code>e.value</code> 为当前状态）
close_on_click	bool	点击菜单项后是否关闭菜单（默认 True，点击后关闭；设为 False 时保持打开）
offset	Tuple[int, int]	菜单显示位置偏移量（单位：像素），格式为 <code>(x_offset, y_offset)</code> （默认无偏移）
max_height	str int	菜单最大高度（支持像素值如 <code>300</code> 、百分比如 <code>'80%'</code> ，超出部分滚动）

配置示例

```
# 右键触发、点击菜单项不关闭、偏移显示、最大高度 200px 的菜单
def on_menu_toggle(e):
    ui.notify(f'菜单已{"显示" if e.value else "隐藏"}')

with ui.button('右键菜单') as trigger:
    with ui.menu(
        trigger=trigger,
        on_context_menu=True,
        close_on_click=False,
        offset=(10, 10),
        max_height=200,
        on_value_change=on_menu_toggle
    ):
        ui.menu_item('选项1', on_click=lambda: ui.notify('选择选项1'))
        ui.menu_item('选项2', on_click=lambda: ui.notify('选择选项2'))
        ui.menu_item('选项3', on_click=lambda: ui.notify('选择选项3'))
        # 超出最大高度的菜单项会自动滚动
        for i in range(4, 11):
            ui.menu_item(f'选项{i}', on_click=lambda i=i: ui.notify(f'选择选项{i}'))
```

三、核心属性

`ui.menu` 继承 NiceGUI 基础元素的通用属性，支持样式配置、状态绑定等，关键属性如下（含可设置属性）：

属性名	类型	说明
classes	str	元素的 CSS 类名（支持 Tailwind、Quasar 类，如 <code>w-48</code> 固定宽度、 <code>bg-gray-50</code> 背景色）
props	str	Quasar 组件属性（用于扩展功能，如 <code>color=primary</code> 设置颜色、 <code>rounded</code> 圆角、 <code>shadow</code> 阴影）
style	str	内联 CSS 样式（如 <code>font-size: 14px;</code> 调整字体大小）
value	BindableProperty	菜单显示状态（布尔值， <code>True</code> 显示、 <code>False</code> 隐藏，支持双向绑定）
visible	BindableProperty	元素可见性（布尔值，支持动态绑定，与 <code>value</code> 区别： <code>visible</code> 控制元素是否存在， <code>value</code> 控制菜单是否展开）
enabled	BindableProperty	菜单是否启用（布尔值， <code>False</code> 时无法触发显示）
trigger	Element Page（可设置）	动态修改触发元素（如 <code>menu.trigger = new_button</code> ）
on_context_menu	bool（可设置）	动态切换触发方式（如 <code>menu.on_context_menu = True</code> 改为右键触发）
close_on_click	bool（可设置）	动态修改点击菜单项后是否关闭菜单
offset	Tuple [int, int]（可设置）	动态修改偏移量（如 <code>menu.offset = (20, 20)</code> ）
max_height	str int（可设置）	动态修改最大高度（如 <code>menu.max_height = 300</code> ）
html_id	str	HTML DOM 中的元素 ID（版本 2.16.0 新增）
is_deleted	bool	元素是否已被删除（只读属性）
parent_slot	Slot None	父容器的插槽（可手动设置）

属性使用示例

```
# 自定义样式的菜单：固定宽度、圆角、阴影、灰色背景
with ui.button('自定义样式菜单') as trigger:
    menu = ui.menu(trigger=trigger).props('rounded shadow color=blue').classes('w-56 bg-gray-50')
    ui.menu_item('新建项目').props('icon=add') # 带图标的菜单项
    ui.menu_item('打开项目').props('icon=folder_open')
    ui.menu_divider()
    ui.menu_item('设置').props('icon=settings')

# 动态修改菜单属性
def update_menu():
    menu.on_context_menu = not menu.on_context_menu # 切换触发方式
    menu.offset = (15, 15) if menu.offset == (0, 0) else (0, 0) # 切换偏移
    ui.notify(f'右键触发: {menu.on_context_menu}, 偏移: {menu.offset}')
```

```
ui.button('切换菜单配置', on_click=update_menu).classes('mt-2')
```

四、核心方法

ui.menu 提供丰富的方法用于控制菜单显示 / 隐藏、绑定状态、管理元素等，常用方法分类如下：

1. 菜单状态控制方法

方法名	作用	示例
show()	手动显示菜单	<code>menu.show()</code>
hide()	手动隐藏菜单	<code>menu.hide()</code>
toggle()	切换菜单显示 / 隐藏状态（显示→隐藏，隐藏→显示）	<code>menu.toggle()</code>
set_value(value: bool)	设置菜单显示状态（True 显示，False 隐藏）	<code>menu.set_value(True)</code>
enable()	启用菜单（允许触发显示）	<code>menu.enable()</code>
disable()	禁用菜单（禁止触发显示）	<code>menu.disable()</code>
set_enabled(value: bool)	动态设置启用状态	<code>menu.set_enabled(False)</code>

2. 绑定方法

支持将菜单显示状态、启用状态、可见性与外部变量绑定，支持单向 / 双向绑定，核心方法如下：

方法名	作用	关键参数
bind_value(target_object, target_name='value')	双向绑定菜单显示状态到目标对象的属性	target_object：绑定目标对象； target_name：绑定的属性名（默认 'value'）
bind_value_from(...)	单向绑定（从目标对象同步到菜单）	同 bind_value，仅单向同步（目标对象属性变化触发菜单显示状态变化）
bind_value_to(...)	单向绑定（从菜单同步到目标对象）	同 bind_value，仅单向同步（菜单显示状态变化触发目标对象属性变化）
bind_enabled(...)	双向绑定启用状态到目标对象的属性	同 bind_value，绑定的属性为启用状态（默认 'enabled'）
bind_visibility(...)	双向绑定可见性到目标对象的属性	value：可选，指定目标值匹配时才显示菜单元素

3. 元素管理方法

方法名	作用	参数说明
clear()	删除菜单内所有子元素（菜单项、子菜单、分隔线）	无参数
remove(element)	删除指定子元素	element: 子元素实例 (<code>ui.menu_item</code> / <code>ui.submenu</code> / <code>ui.menu_divider</code> 对象) 或其 ID
add_item(label: str, on_click: Callable = None, **kwargs)	快速添加菜单项	label: 菜单项文本; on_click: 点击回调; **kwargs: 其他参数 (如 <code>icon</code> 、 <code>props</code>)
add_divider()	快速添加菜单分隔线	无参数
delete()	删除整个菜单元素及所有子元素	无参数
move(target_container=None, target_index=-1, target_slot=None)	移动菜单到其他容器	target_container: 目标容器; target_index: 目标索引; target_slot: 目标插槽

4. 其他常用方法

方法名	作用	示例
tooltip(text)	为菜单触发元素添加鼠标悬浮提示（菜单本身无 tooltip，需绑定到触发元素）	<code>trigger.tooltip('点击打开菜单')</code>
update()	强制更新客户端的菜单状态（如动态添加菜单项后刷新）	<code>menu.update()</code>
add_resource(path)	为菜单添加资源文件（如自定义 CSS、JS）	<code>menu.add_resource('./static')</code>
ancestors(include_self=False)	迭代获取所有祖先元素	遍历祖先元素: <code>for elem in menu.ancestors(): print(elem)</code>
mark(*markers)	为元素添加标记（用于测试或元素查询）	<code>menu.mark('context-menu', '2024')</code>

方法使用示例

```
from nicegui import ui

# 绑定菜单显示状态到外部变量
class MenuState:
    is_menu_open = False

state = MenuState()

with ui.button('绑定状态菜单') as trigger:
    menu = ui.menu(trigger=trigger).bind_value(state, 'is_menu_open')
    ui.menu_item('选项1', on_click=lambda: ui.notify('选择选项1'))
    ui.menu_item('选项2', on_click=lambda: ui.notify('选择选项2'))

# 显示绑定的状态
ui.label('菜单状态: ').bind_text_from(state, 'is_menu_open', lambda s: '打开' if s else '关闭').classes('mt-2')

# 程序控制菜单
ui.row(
    ui.button('显示菜单', on_click=menu.show),
    ui.button('隐藏菜单', on_click=menu.hide),
    ui.button('切换菜单', on_click=menu.toggle),
    ui.button('添加菜单项', on_click=lambda: menu.add_item('动态添加选项', on_click=lambda: ui.notify('动态选项被点击'))).classes('mt-2')

ui.run()
```

五、菜单子元素详解

ui.menu 的核心子元素包括 `ui.menu_item` (菜单项)、`ui.submenu` (子菜单)、`ui.menu_divider` (分隔线)，各自配置如下：

1. ui.menu_item (菜单项)

初始化参数

参数名	类型	说明
label	str	菜单项文本 (必填)
on_click	Callable	点击菜单项时触发的回调函数
icon	str	菜单项左侧图标 (支持 Quasar 图标库名称, 如 'add'、'delete')
props	str	Quasar 组件属性 (如 <code>color=red</code> 红色文本、 <code>disabled</code> 禁用状态、 <code>active</code> 激活状态)
classes	str	CSS 类名 (如 <code>text-bold</code> 加粗文本、 <code>py-2</code> 调整内边距)

使用示例

```
# 带图标、禁用状态、激活状态、自定义颜色的菜单项
ui.menu_item('正常选项', icon='check', on_click=lambda: ui.notify('正常选项'))
ui.menu_item('禁用选项', icon='block', on_click=lambda: ui.notify('禁用选项')).props('disabled')
ui.menu_item('激活选项', icon='star', on_click=lambda: ui.notify('激活选项')).props('active')
ui.menu_item('红色选项', icon='warning', on_click=lambda: ui.notify('红色选项')).props('color=red')
```

2. ui.submenu (子菜单)

初始化参数

参数名	类型	说明
label	str	子菜单触发文本（必填）
icon	str	子菜单左侧图标（支持 Quasar 图标库名称）
props	str	Quasar 组件属性（如 color=blue、rounded）
classes	str	CSS 类名（如 bg-gray-100 背景色）

使用示例

```
# 多级子菜单嵌套
with ui.submenu('文件', icon='folder'):
    ui.menu_item('新建', icon='add')
    ui.menu_item('打开', icon='folder_open')
    with ui.submenu('导入', icon='import_contacts'): # 二级子菜单
        ui.menu_item('从本地导入', icon='file_upload')
        ui.menu_item('从云端导入', icon='cloud_download')
    ui.menu_divider()
    ui.menu_item('退出', icon='exit_to_app').props('color=red')
```

3. ui.menu_divider (菜单分隔线)

初始化参数

参数名	类型	说明
props	str	Quasar 组件属性（如 color=gray-300 调整分隔线颜色）
classes	str	CSS 类名（如 my-1 调整上下边距）

使用示例

```
# 自定义颜色和边距的分隔线
ui.menu_item('选项1')
ui.menu_divider(props='color=blue-200').classes('my-2')
ui.menu_item('选项2')
```

六、事件处理

1. 核心事件：菜单显示 / 隐藏状态变化事件

通过 `on_value_change` 绑定状态变化回调，获取当前显示状态：

```
def on_menu_state_change(e):
    ui.notify(f'菜单状态: {"显示" if e.value else "隐藏"}')

with ui.button('状态监听菜单') as trigger:
    ui.menu(trigger=trigger, on_value_change=on_menu_state_change)
    ui.menu_item('选项1')
```

2. 菜单项点击事件

通过 `ui.menu_item` 的 `on_click` 参数绑定，或通过 `on()` 方法绑定：

```
# 方式1: 初始化时绑定
ui.menu_item('点击回调', on_click=lambda: ui.notify('菜单项被点击'))

# 方式2: 通过 on() 方法绑定
item = ui.menu_item('动态绑定回调')
item.on('click', lambda: ui.notify('动态绑定的点击事件'))
```

3. 通用事件绑定

为菜单容器绑定 DOM 事件（如鼠标悬浮、离开），支持 Python 或 JavaScript 回调：

```
with ui.button('事件绑定菜单') as trigger:
    menu = ui.menu(trigger=trigger)
    ui.menu_item('选项1')

# 菜单容器鼠标悬浮事件
menu.on('mouseover', lambda: ui.notify('鼠标进入菜单'))
# 菜单容器鼠标离开事件
menu.on('mouseout', lambda: ui.notify('鼠标离开菜单'))
```


七、高级用法

1. 动态菜单管理

根据业务逻辑动态添加、删除菜单项或子菜单，示例：

```
from nicegui import ui

with ui.button('动态菜单') as trigger:
    menu = ui.menu(trigger=trigger)
    # 初始菜单项
    menu.add_item('初始选项1', on_click=lambda: ui.notify('初始选项1'))
    menu.add_item('初始选项2', on_click=lambda: ui.notify('初始选项2'))
    menu.add_divider()

# 动态添加菜单项
def add_menu_item():
    menu.add_item(f'动态选项{len(menu.default_slot.children)}', on_click=lambda
i=len(menu.default_slot.children): ui.notify(f'动态选项{i}'))

# 动态添加子菜单
def add_submenu():
    with menu:
        with ui.submenu(f'动态子菜单{len(menu.default_slot.children)}',
icon='subdirectory_arrow_right'):
            ui.menu_item('子选项1')
            ui.menu_item('子选项2')

# 清空菜单
def clear_menu():
    menu.clear()
    ui.notify('菜单已清空')

ui.row(
    ui.button('添加菜单项', on_click=add_menu_item),
    ui.button('添加子菜单', on_click=add_submenu),
    ui.button('清空菜单', on_click=clear_menu)
).classes('mt-2')

ui.run()
```

2. 菜单与数据联动（实战场景）

结合表格数据，为每行数据添加右键菜单，实现快捷操作：

```
from nicegui import ui

# 模拟表格数据
data = [{'name': '文件1', 'size': '100KB'}, {'name': '文件2', 'size': '200KB'}, {'name': '文件
3', 'size': '300KB'}]

# 创建表格
```

```

table = ui.table(columns=[{'name': 'name', 'label': '文件名', 'field': 'name'}, {'name': 'size', 'label': '大小', 'field': 'size'}], rows=data)

# 为表格行添加右键菜单
def create_row_menu(row_index):
    def show_menu(e):
        # 阻止默认右键菜单
        e.preventDefault()
        # 创建临时菜单（点击后自动销毁）
        with ui.menu(trigger=ui.page, on_context_menu=False) as menu:
            ui.menu_item(f'打开 {data[row_index]["name"]}', on_click=lambda: ui.notify(f'打开文件: {data[row_index]["name"]}'))
            ui.menu_item(f'删除 {data[row_index]["name"]}', on_click=lambda: ui.notify(f'删除文件: {data[row_index]["name"]}'))
            ui.menu_item(f'重命名 {data[row_index]["name"]}', on_click=lambda: ui.notify(f'重命名文件: {data[row_index]["name"]}'))
        # 显示菜单在鼠标位置
        menu.show()

    return show_menu

# 为表格每行绑定右键事件
for i, row in enumerate(data):
    table.rows[i].on('contextmenu', create_row_menu(i))

ui.run()

```

3. 样式深度自定义

结合 Quasar props 和 Tailwind 类，实现个性化菜单样式：

```

from nicegui import ui

with ui.button('深度自定义菜单', classes='bg-purple-600 text-white') as trigger:
    with ui.menu(trigger=trigger).props('rounded-lg shadow-lg color=purple').classes('w-64 bg-purple-50 border border-purple-100'):
        # 自定义菜单项样式
        ui.menu_item('首页').props('icon=home color=purple hover:bg-purple-100').classes('py-3')
        ui.menu_item('设置').props('icon=settings color=purple hover:bg-purple-100').classes('py-3')
        ui.menu_divider(props='color=purple-200').classes('my-1')
        # 子菜单自定义样式
        with ui.submenu('账户', icon='account_circle', props='color=purple hover:bg-purple-100').classes('py-3'):
            ui.menu_item('个人资料').props('hover:bg-purple-100')
            ui.menu_item('退出登录').props('color=red hover:bg-red-100')

ui.run()

```

八、注意事项

- 1. 触发元素约束：`trigger` 参数必须是有效的 NiceGUI 元素或 `ui.page`，否则菜单无法触发显示；若动态修改 `trigger`，需确保新触发元素已挂载到 DOM。
- 2. 右键触发兼容性：`on_context_menu=True` 时，需注意与浏览器默认右键菜单冲突，可通过 `e.preventDefault()` 阻止默认行为。
- 3. 子菜单嵌套深度：支持多级子菜单嵌套，但建议不超过 3 级，避免层级过深影响用户体验。
- 4. 菜单显示位置：菜单默认自动贴合触发元素，若触发元素在页面边缘，菜单会自动调整位置以确保完全显示；若需手动调整，可通过 `offset` 参数设置。
- 5. 动态更新后刷新：动态添加 / 删除菜单项或修改菜单属性后，建议调用 `update()` 方法刷新客户端显示，确保状态同步。
- 6. 版本兼容性：`html_id` 属性需 NiceGUI 2.16.0+ 版本支持，`bind_enabled` 方法需 3.0.0+ 版本支持，使用时需确认版本匹配。

通过以上配置与方法，`ui.menu` 可灵活满足从简单下拉菜单到复杂上下文菜单的各类需求，是 NiceGUI 中实现界面操作导航的核心组件之一。

ui.menu_item 全面详解

`ui.menu_item` 是 NiceGUI 中配合 `ui.menu` 使用的核心子元素，用于定义菜单中的单个可交互选项，支持点击回调、图标配置、状态控制（启用 / 禁用）、样式自定义等功能，是构建菜单结构的基础组件。其核心特性包括轻量化交互、灵活的样式配置、状态绑定等，广泛适用于各类菜单（下拉菜单、上下文菜单、子菜单）的选项定义。

一、核心概念与基础特性

1. 本质与用途

- 本质：Quasar 组件的封装元素，作为 `ui.menu` 或 `ui.submenu` 的直接子元素，承载单个菜单选项的文本、图标、点击事件等核心信息。
- 核心用途：提供菜单中的可点击选项，触发具体业务逻辑（如“新建文件”“删除数据”“跳转页面”等），是菜单与用户交互的核心载体。
- 关键机制：默认点击后关闭所属菜单（可通过参数禁用），支持配置图标、颜色、禁用状态等，可与外部变量绑定实现状态动态同步。

2. 基础结构

`ui.menu_item` 需嵌套在 `ui.menu` 或 `ui.submenu` 容器中，基础使用示例如下：

```
from nicegui import ui

with ui.button('操作菜单'):
    with ui.menu():
        # 基础菜单项：仅文本+点击回调
        ui.menu_item('新建文件', on_click=lambda: ui.notify('新建文件成功'))
        # 带图标的菜单项
        ui.menu_item('打开文件', on_click=lambda: ui.notify('打开文件成功'),
            icon='folder_open')
```

```
        # 禁用状态的菜单项
        ui.menu_item('删除文件', on_click=lambda: ui.notify('删除文件成功'),
icon='delete').props('disabled')
        # 自定义颜色的菜单项
        ui.menu_item('退出程序', on_click=lambda: ui.notify('退出程序'),
icon='exit_to_app').props('color=red')

ui.run()
```

二、初始化配置项

初始化 `ui.menu_item()` 时可通过参数配置文本、图标、回调、状态等核心属性，参数说明如下：

参数名	类型	说明
text	str	菜单项的显示文本（必填，支持普通字符串或带换行的多行文本）
on_click	Callable	点击菜单项时触发的回调函数（可选，无回调时菜单项仅作为展示，无交互）
auto_close	bool	点击后是否关闭所属菜单（默认 True，设为 False 时保持菜单打开，适用于子菜单触发）
icon	str	菜单项左侧的图标（可选，支持 Quasar 图标库名称，如 'rocket'、'check'）

配置示例

```
from nicegui import ui

with ui.button('高级菜单'):
    with ui.menu():
        # 点击不关闭菜单的菜单项（适用于多选逻辑）
        ui.menu_item('选项1（不关闭）', on_click=lambda: ui.notify('选中选项1'),
auto_close=False)
        ui.menu_item('选项2（不关闭）', on_click=lambda: ui.notify('选中选项2'),
auto_close=False)
        # 带图标+自定义文本的菜单项
        ui.menu_item('查看详情', on_click=lambda: ui.notify('查看详情'), icon='info')
        # 无回调的纯展示菜单项
        ui.menu_item('-- 分隔文本 --')

ui.run()
```

三、核心属性

`ui.menu_item` 继承 NiceGUI 基础元素的通用属性，支持样式、状态、绑定等配置，关键属性如下：

属性名	类型	说明
classes	str	元素的 CSS 类名（支持 Tailwind、Quasar 类，如 <code>text-bold</code> 加粗、 <code>py-2</code> 调整内边距）

属性名	类型	说明
props	str	Quasar 组件属性 (如 <code>disabled</code> 禁用、 <code>color=blue</code> 文本颜色、 <code>active</code> 激活状态、 <code>flat</code> 扁平化样式)
style	str	内联 CSS 样式 (如 <code>font-size: 14px;</code> 调整字体大小、 <code>padding: 8px 16px;</code> 调整内边距)
enabled	BindableProperty	启用状态 (布尔值, True 为启用, False 为禁用, 支持双向绑定)
visible	BindableProperty	元素可见性 (布尔值, 支持动态绑定, True 显示、False 隐藏)
html_id	str	HTML DOM 中的元素 ID (版本 2.16.0 新增, 用于精准定位)
is_deleted	bool	元素是否已被删除 (只读属性, 用于判断元素状态)
parent_slot	Slot None	父容器的插槽 (可手动设置元素所属的父插槽, 用于复杂布局嵌套)

属性使用示例

```
from nicegui import ui

with ui.button('样式自定义菜单'):
    with ui.menu():
        # 加粗+蓝色文本+扁平化样式的菜单项
        ui.menu_item('加粗蓝色选项', icon='check').props('color=blue flat').classes('text-bold')
        # 自定义内边距+字体大小的菜单项
        ui.menu_item('自定义样式选项', icon='settings').style('padding: 10px 20px; font-size: 15px;')
        # 激活状态的菜单项 (常用于当前选中选项)
        ui.menu_item('当前选中选项', icon='star').props('active')

ui.run()
```

四、核心方法

ui.menu_item 提供丰富的方法用于控制状态、绑定变量、管理元素等，常用方法分类如下：

1. 状态控制方法

方法名	作用	示例
enable()	启用菜单项 (启用后可点击触发回调)	<code>menu_item.enable()</code>
disable()	禁用菜单项 (禁用后不可点击, 样式变灰)	<code>menu_item.disable()</code>
set_enabled(value: bool)	动态设置启用状态 (True 启用, False 禁用)	<code>menu_item.set_enabled(False)</code>

方法名	作用	示例
set_visibility(visible: bool)	设置可见性（True 显示, False 隐藏）	<code>menu_item.set_visibility(False)</code>
on_click(callback: Callable)	动态绑定点击回调（覆盖初始配置的 on_click）	<code>menu_item.on_click(lambda: ui.notify('动态绑定回调'))</code>

2. 绑定方法

支持将启用状态、可见性与外部变量绑定，实现状态同步，核心方法如下：

方法名	作用	关键参数
bind_enabled(target_object, target_name='enabled')	双向绑定启用状态到目标对象的属性	target_object: 绑定目标对象; target_name: 绑定的属性名（默认 'enabled'）
bind_enabled_from(...)	单向绑定（从目标对象同步到菜单项）	同 bind_enabled，仅单向同步（目标对象属性变化触发菜单项启用状态变化）
bind_enabled_to(...)	单向绑定（从菜单项同步到目标对象）	同 bind_enabled，仅单向同步（菜单项启用状态变化触发目标对象属性变化）
bind_visibility(...)	双向绑定可见性到目标对象的属性	value: 可选，指定目标值匹配时才显示菜单项

3. 其他常用方法

方法名	作用	示例
tooltip(text: str)	为菜单项添加鼠标悬浮提示	<code>menu_item.tooltip('点击执行操作')</code>
update()	强制更新客户端的菜单项状态（如动态修改属性后刷新）	<code>menu_item.update()</code>
delete()	删除当前菜单项	<code>menu_item.delete()</code>
mark(*markers: str)	为元素添加标记（用于测试或元素查询）	<code>menu_item.mark('action-item', 'delete')</code>

方法使用示例

```
from nicegui import ui

# 绑定菜单项状态到外部变量
class ItemState:
    is_enabled = True
```

```

is_visible = True

state = ItemState()

with ui.button('状态绑定菜单'):
    with ui.menu():
        menu_item = ui.menu_item(
            '动态状态选项',
            on_click=lambda: ui.notify('选项被点击'),
            icon='dynamic_feed'
        )
        # 双向绑定启用状态和可见性
        menu_item.bind_enabled(state, 'is_enabled')
        menu_item.bind_visibility(state, 'is_visible')

# 控制按钮：切换启用状态
ui.button('切换启用/禁用', on_click=lambda: setattr(state, 'is_enabled', not
state.is_enabled)).classes('mt-2')
# 控制按钮：切换可见性
ui.button('切换显示/隐藏', on_click=lambda: setattr(state, 'is_visible', not
state.is_visible)).classes('ml-2')
# 控制按钮：动态绑定新回调
ui.button('绑定新回调', on_click=lambda: menu_item.on_click(lambda: ui.notify('动态更新的回
调'))).classes('ml-2')

ui.run()

```

五、事件处理

1. 核心事件：点击事件

点击事件是 `ui.menu_item` 的核心交互事件，支持两种绑定方式：

```

from nicegui import ui

with ui.button('事件绑定菜单'):
    with ui.menu():
        # 方式1：初始化时绑定
        ui.menu_item('初始绑定回调', on_click=lambda: ui.notify('初始回调触发'))

        # 方式2：通过 on() 方法动态绑定（支持绑定多个事件）
        item = ui.menu_item('动态绑定回调', icon='event')
        item.on('click', lambda e: ui.notify('动态回调触发（事件参数：%s）' % e))

ui.run()

```

2. 其他通用事件

支持绑定 DOM 通用事件（如鼠标悬浮、鼠标离开），示例：

```
from nicegui import ui

with ui.button('通用事件菜单'):
    with ui.menu():
        item = ui.menu_item('鼠标交互选项', icon='mouse')
        # 鼠标悬浮事件
        item.on('mouseover', lambda: ui.notify('鼠标进入菜单项'))
        # 鼠标离开事件
        item.on('mouseout', lambda: ui.notify('鼠标离开菜单项'))

ui.run()
```

六、高级用法

1. 与子菜单配合使用

`ui.menu_item` 可作为子菜单的触发元素（需设置 `auto_close=False`），嵌套 `ui.menu` 实现多级菜单：

```
from nicegui import ui

with ui.button('多级菜单'):
    with ui.menu():
        ui.menu_item('基础操作', icon='settings')
        # 子菜单触发项：点击不关闭父菜单
        with ui.menu_item('更多操作', auto_close=False) as submenu_trigger:
            with ui.item_section().props('side'):
                ui.icon('keyboard_arrow_right') # 右侧箭头标识子菜单
            # 子菜单内容
            with ui.menu().props('anchor="top end" self="top start" auto-close'):
                ui.menu_item('子选项1', on_click=lambda: ui.notify('子选项1被点击'))
                ui.menu_item('子选项2', on_click=lambda: ui.notify('子选项2被点击'))

ui.run()
```

2. 动态修改菜单项属性

结合业务逻辑动态修改文本、图标、样式等属性，示例：

```
from nicegui import ui

with ui.button('动态菜单'):
    with ui.menu() as menu:
        dynamic_item = ui.menu_item('初始文本', icon='edit', on_click=lambda: ui.notify('初始文本'))

# 动态修改文本和图标
def update_text_icon():
```



```

dynamic_item.text = '修改后的文本'
dynamic_item.icon = 'done'
dynamic_item.update() # 刷新客户端显示
ui.notify('菜单项文本和图标已更新')

# 动态修改样式
def update_style():
    dynamic_item.props('color=green flat')
    dynamic_item.classes('text-bold')
    dynamic_item.update()
    ui.notify('菜单项样式已更新')

ui.row(
    ui.button('更新文本和图标', on_click=update_text_icon),
    ui.button('更新样式', on_click=update_style)
).classes('mt-2')

ui.run()

```

3. 结合其他组件实现复杂交互

菜单项中可嵌套其他 NiceGUI 元素（如开关、输入框），实现复杂交互逻辑：

```

from nicegui import ui

with ui.button('复杂交互菜单'):
    with ui.menu().props('width=300px'):
        # 菜单项中嵌套开关
        with ui.menu_item(auto_close=False):
            ui.switch('启用功能', on_change=lambda e: ui.notify(f'功能已{"启用" if e.value else "禁用"}'))
        # 菜单项中嵌套输入框
        with ui.menu_item(auto_close=False):
            ui.input('输入备注', placeholder='请输入备注信息').on('input', lambda e: print('输入内容: ', e.value))
        # 普通交互菜单项
        ui.menu_item('提交', on_click=lambda: ui.notify('提交成功')).props('color=blue')

ui.run()

```

七、注意事项

1. 菜单嵌套约束：作为子菜单触发元素时，必须设置 `auto_close=False`，否则点击后父菜单会关闭，子菜单无法显示；同时需配合 `ui.item_section` 添加右侧箭头，提升用户体验。
2. 图标兼容性：`icon` 参数仅支持 Quasar 图标库中的名称，需确保图标名称正确，否则不会显示图标（无报错提示）。
3. 禁用状态影响：禁用（`disabled`）的菜单项会自动屏蔽 `on_click` 回调，即使动态绑定新回调也无法触发，需先启用（`enable()`）再绑定。
4. 文本换行：若需多行文本，可在 `text` 中使用 `\n` 换行，配合 `white-space: pre-line` 样式实现换行显示。

5. 版本兼容性: `html_id` 属性需 NiceGUI 2.16.0+ 版本支持, `bind_enabled` 的 `strict` 参数需 3.0.0+ 版本支持, 使用时需确认版本匹配。

通过以上配置与方法, `ui.menu_item` 可灵活满足从简单文本选项到复杂交互选项的各类需求, 是构建功能完善、交互友好菜单的核心基础组件。

NiceGUI 的 ui.submenu 详细解析

`ui.submenu` 是 NiceGUI 中用于创建**嵌套子菜单**的核心组件, 通常配合 `ui.menu` (主菜单) 或其他容器使用, 用于构建层级化的菜单结构, 常见于导航栏、右键菜单、下拉菜单等场景。

一、核心特性

- 1. **层级嵌套**: 支持多层级子菜单 (子菜单内可再嵌套子菜单) ;
- 2. **触发方式**: 默认鼠标悬停触发展开, 也可配置点击触发;
- 3. **样式自定义**: 支持修改标签、图标、颜色、尺寸等样式;
- 4. **事件绑定**: 可给子菜单项绑定点击、hover 等事件;
- 5. **动态控制**: 支持运行时动态添加 / 删除子菜单项、显示 / 隐藏子菜单。

二、基本用法

`ui.submenu` 必须作为 `ui.menu` 或其他 `ui.submenu` 的子元素使用, 基础语法如下:

```
from nicegui import ui

# 1. 创建主菜单容器 (以导航栏为例)
with ui.header():
    with ui.menu_button('主菜单'): # 触发主菜单的按钮
        with ui.menu(): # 主菜单容器
            # 2. 创建一级子菜单
            with ui.submenu('一级子菜单'):
                with ui.menu(): # 子菜单的内容容器
                    ui.menu_item('子菜单项1', on_click=lambda: print('点击项1'))
                    ui.menu_item('子菜单项2')
            # 3. 嵌套二级子菜单
            with ui.submenu('二级子菜单', icon='settings'):
                with ui.menu():
                    ui.menu_item('二级项1')
                    ui.menu_item('二级项2')

            # 主菜单的普通项
            ui.menu_item('主菜单普通项')

ui.run()
```

三、关键参数说明

参数名	类型	作用
<code>text</code>	str	子菜单的显示文本 (必填)

参数名	类型	作用
<code>icon</code>	str	子菜单左侧图标（支持 Font Awesome 图标名，如 'home'、'settings'）
<code>color</code>	str	文本 / 图标颜色（支持 CSS 颜色值，如 'red'、'#ff0000'、'rgb (255,0,0)'
<code>on_click</code>	Callable	点击子菜单标题时的回调（默认悬停展开，设置后可改为点击展开）
<code>disabled</code>	bool	是否禁用子菜单（禁用后无法展开，样式变灰）
<code>props</code>	str/list	传递给底层 Quasar 组件的属性（如 'dense' 缩小尺寸、'bordered' 加边框）

四、高级用法

1. 点击触发展开（替代默认悬停）

默认子菜单是鼠标悬停展开，可通过绑定 `on_click` + 手动控制展开状态实现点击触发：

```
from nicegui import ui

with ui.menu_button('主菜单'):
    with ui.menu() as main_menu:
        # 创建子菜单并记录展开状态
        submenu_expanded = ui.ref(False)
        with ui.submenu('点击展开子菜单', on_click=lambda: submenu_expanded.set(not submenu_expanded.value)) as sm:
            with ui.menu():
                ui.menu_item('项1')
                ui.menu_item('项2')
            # 监听展开状态，控制子菜单显示
            sm.bind_visible(submenu_expanded)

ui.run()
```

2. 动态添加子菜单项

运行时可通过 `ui.menu` 的容器特性动态添加项：

```
from nicegui import ui

def add_item():
    # 向子菜单的内容容器中添加新项
    submenu_content.add(ui.menu_item(f'动态项{len(submenu_content.children)+1}'))

with ui.menu_button('主菜单'):
    with ui.menu():
        # 保存子菜单的内容容器引用
        with ui.submenu('动态子菜单'):
            submenu_content = ui.menu()
            ui.menu_item('初始项1')
        # 外部按钮触发添加
    ui.button('添加子菜单项', on_click=add_item)
```

```
ui.run()
```

3. 自定义样式

通过 `props` 和 `style` 自定义子菜单外观：

```
with ui.menu_button('主菜单'):
    with ui.menu():
        # 自定义子菜单：紧凑样式 + 红色文本 + 边框
        with ui.submenu('自定义样式子菜单', color='red', props=['dense', 'bordered'],
style='font-weight: bold;'):
            with ui.menu(style='background: #f5f5f5;'): # 子菜单背景色
                ui.menu_item('项1', style='color: black;')
```

五、注意事项

1. `ui.submenu` 必须包裹在 `ui.menu` 或其他 `ui.submenu` 内，不能直接独立使用；
2. 多层嵌套子菜单建议控制层级（不超过 3 层），避免交互体验变差；
3. 底层依赖 Quasar 的 `q-menu` 和 `q-submenu` 组件，复杂 props 可参考[Quasar 文档](#)；
4. 禁用子菜单时，需同时禁用其内部所有项，避免逻辑冲突。

NiceGUI 中 `ui.menu_divider` 的详细解析

`ui.menu_divider` 是 NiceGUI 框架中用于在菜单（`ui.menu` / `ui.context_menu` 等）内添加分隔线的组件，核心作用是对菜单项进行分组，提升菜单的视觉层次感和交互体验。

一、核心特性

1. **功能定位**：纯视觉组件，无交互逻辑，仅用于分隔不同类别的菜单项，无法绑定点击事件、设置禁用状态等。
2. **适配场景**：仅能在 `ui.menu`、`ui.context_menu`（右键菜单）、`ui.dropdown`（下拉菜单）等菜单类容器中使用，在普通容器（如 `ui.card`、`ui.row`）中使用会无效果或报错。
3. **样式特性**：默认继承 NiceGUI 的主题样式（颜色、高度、边距），也可通过自定义 CSS 调整外观。

二、基本用法

1. 基础示例（菜单内分隔菜单项）

```
from nicegui import ui

# 普通菜单+分隔线
with ui.menu() as menu:
    ui.menu_item('新建文件')
    ui.menu_item('打开文件')
    ui.menu_divider() # 分隔线
    ui.menu_item('保存')
    ui.menu_item('另存为')
    ui.menu_divider() # 再次分隔
    ui.menu_item('退出')

# 触发按钮
```

```
ui.button('打开菜单', on_click=menu.open)

ui.run()
```

2. 右键菜单中使用

```
from nicegui import ui

# 右键菜单+分隔线
with ui.context_menu() as ctx_menu:
    ui.menu_item('复制')
    ui.menu_item('粘贴')
    ui.menu_divider()
    ui.menu_item('删除')
    ui.menu_item('重命名')

# 绑定到文本框
ui.input('右键点击我').bind_context_menu(ctx_menu)

ui.run()
```

3. 下拉菜单中使用

```
from nicegui import ui

# 下拉菜单+分隔线
with ui.dropdown('操作') as dropdown:
    ui.menu_item('导出Excel')
    ui.menu_item('导出PDF')
    ui.menu_divider()
    ui.menu_item('导入数据')
    ui.menu_item('清空数据')

ui.run()
```

三、自定义样式

通过 `style` 参数或全局 CSS 修改分隔线外观：

```
from nicegui import ui

with ui.menu():
    ui.menu_item('基础功能1')
    # 自定义分隔线：红色、加粗、增加高度
    ui.menu_divider(style='border-top: 2px solid red; margin: 4px 0;')
    ui.menu_item('高级功能1')

ui.run()
```

四、注意事项

1. `ui.menu_divider` 无需传入任何参数，传入参数会被忽略；
2. 分隔线的显示效果依赖父菜单容器的布局，若菜单无足够高度，分隔线可能被压缩；
3. 与 `ui.separator`（通用分隔线）的区别：`ui.menu_divider` 是为菜单优化的轻量级分隔线，样式更贴合菜单场景；`ui.separator` 是通用分隔线，可用于任意容器，但在菜单中使用会破坏菜单的默认布局。